

Day 13: Service Mesh, Anthos, and End-to-End Microservices

Part 1: Service Mesh Deep Dive

1.1 Formal Definition

A **Service Mesh** is a dedicated infrastructure layer that controls how different parts of an application share data with one another. Unlike a traditional library, it is integrated into the platform itself, abstracting away the logic required for communication, security, and reliability.

1.2 The "Sidecar" Proxy Pattern (Envoy)

In a mesh, every microservice is paired with a **Sidecar Proxy**.

- **The Application:** Focuses purely on business logic (e.g., "process payment").
- **The Proxy:** Handles all networking tasks (e.g., "encrypt this data," "retry if the server is busy," "report latency to the dashboard").
- **Result:** Developers no longer need to write networking code for every language/framework used in the company.

1.3 Critical Capabilities

- **mTLS (Mutual TLS):** Automatically issues and rotates certificates so that every service authenticates every other service.
- **Circuit Breaking:** If a service starts failing, the mesh stops sending traffic to it temporarily to prevent a total system crash (cascading failure).
- **Fault Injection:** Testing resilience by artificially delaying 10% of requests to see if the system recovers.

Part 2: Anthos and Anthos Service Mesh (ASM)

2.1 What is Anthos?

Anthos is an **Application Management Platform** that provides a consistent environment for both legacy and cloud-native applications. It acts as a "control center" that treats clusters in AWS, Azure, On-prem, and GCP as a single, unified fleet.

2.2 Anthos Service Mesh (ASM)

ASM is Google's managed version of **Istio**. It offloads the "Control Plane" management to Google, meaning you don't have to manage the complex backend that controls the proxies; you only manage the policies.

Part 3: Professional Case Studies

Case Study A: The Global Financial Exchange (Security & Compliance)

- **The Problem:** A financial firm had services spread across three regions. Auditing who accessed which data was impossible because every team logged data differently.
- **The Solution:** They deployed **Anthos Service Mesh**.
- **The Result: * Uniform Logging:** ASM automatically captured "Access Logs" for every single service-to-service call without changing a line of code.
 - **Zero-Trust:** They enforced a policy where only the "Billing" service could talk to the "Credit Card" database, blocking all other internal traffic by default.

Case Study B: E-Commerce "Black Friday" Resilience

- **The Problem:** During high traffic, one slow microservice (the "Recommendations Engine") would hang, causing the entire checkout page to time out.
 - **The Solution:** They implemented **Outlier Detection** and **Timeouts** in ASM.
 - **The Result:** When the Recommendation service slowed down, the Mesh automatically "cut it off" (Circuit Breaking) and served a default response, allowing the checkout process to continue successfully.
-

Part 4: Hands-On Demo - Implementing ASM

Phase 1: Preparation via GCP Console (UI)

1. **Project Selection:** Ensure you are in the correct project.
2. **Enable Fleet Management:** * Navigate to **Anthos > Dashboard**.
 - Click **Enable** to activate the Anthos feature set.
3. **Register Cluster:** * Go to **Kubernetes Engine > Clusters**.
 - Find your cluster, click the three dots, and select **Register to Fleet**. This allows Anthos to manage the cluster.

Phase 2: Installing ASM (CLI Commands)

Use the Google-provided asmcli to install the managed control plane.

Bash

Set project variables

```
export PROJECT_ID=$(gcloud config get-value project)
```

```
export CLUSTER_NAME=production-cluster
```

```
export CLUSTER_LOCATION=us-central1-a
```

Download the installer

```
curl https://storage.googleapis.com/csm-artifacts/asm/asmcli > asmcli
```

```
chmod +x asmcli
```

Install ASM with Managed Control Plane

This installs the components and configures Google-managed certificates

```
./asmcli install \
```

```
--project_id $PROJECT_ID \
```

```
--cluster_name $CLUSTER_NAME \
```

```
--cluster_location $CLUSTER_LOCATION \
```

```
--fleet_id $PROJECT_ID \
```

```
--enable_all \
```

```
--option managed
```

Phase 3: Enabling Sidecar Injection

For the mesh to work, your Pods must have the Envoy proxy injected.

Bash

```
# Identify the revision label created by the installation
```

```
# It usually looks like 'asm-managed'
```

```
kubectl get controlplanerevision -n istio-system
```

```
# Label your namespace (e.g., 'prod') for automatic injection
```

```
kubectl label namespace prod istio.io/rev=asm-managed --overwrite
```

```
# Restart your existing deployments to inject the proxy
```

```
kubectl rollout restart deployment -n prod
```

Phase 4: Traffic Management Example (Canary Release)

Apply this **VirtualService** manifest to send 90% of traffic to your stable version and 10% to your new "Canary" version.

YAML

```
apiVersion: networking.istio.io/v1alpha3
```

```
kind: VirtualService
```

```
metadata:
```

```
  name: product-reviews-route
```

```
spec:
```

```
  hosts:
```

```
    - reviews
```

http:

- route:

- destination:

host: reviews

subset: v1

weight: 90

- destination:

host: reviews

subset: v2

weight: 10

Save this as route.yaml and run `kubectl apply -f route.yaml`.

Part 5: Monitoring and Observability (UI Steps)

Once the mesh is running, you stop using standard logs and start using the **Service Mesh Dashboard**:

1. **Access Dashboard:** In the GCP Console, go to **Anthos > Service Mesh**.
2. **Service List:** You will see a list of all microservices. Notice the **Error Rate** and **Latency** columns.
3. **Topology Map:** Click the **Topology** tab. You will see a live, interactive map of your architecture.
 - **Green Lines:** Healthy communication.
 - **Red Lines:** High error rates between those two specific services.
4. **Health Check:** Click on any specific service (e.g., frontend) to see its **Golden Signals**:
 - **Requests Per Second**

- **Error Rate (%)**
- **Latency (ms)**

This approach ensures that your microservices are not just "running," but are secure, observable, and resilient to failure.